

CYBERRAKSHA

AI Cybersecurity Awareness & Emergency Protection Assistant

IBM Hackathon Project Report

Theme: AI for Cybersecurity Awareness

Tagline: Detect the threat. Explain the danger. Stop the mistake.

Project Snapshot	Details
Problem	Users cannot easily recognize phishing/scam messages and often act under urgency or fear.
Solution	A compact AI assistant that analyzes suspicious messages/screenshots/links/QRs, explains the risk, alerts the user, and applies a Safety Lock before dangerous interaction.
Core Innovation	Explainable Scam Analysis + Emergency Cyber Alert + Safety Lock.
Languages	English, Hindi, Marathi.
Primary users	Students, senior citizens, first-time digital-payment users, and general smartphone users.
MVP principle	Build a focused, reliable prototype instead of a bulky cybersecurity suite.

1. Executive Summary

CyberRaksha is an AI-powered cybersecurity awareness and emergency protection assistant designed around the “Is this safe?” use case. A user can submit a suspicious SMS/WhatsApp message, screenshot, URL or QR code. The system analyzes the content for phishing and scam indicators, produces a risk score and scam category, and explains the danger in simple language. When the threat crosses a high-confidence threshold, CyberRaksha raises an emergency alert and activates a Safety Lock that prevents the user from proceeding to the suspicious destination inside the app. It then recommends the safest next action.

The project deliberately avoids becoming a large security platform. Its value is the short safety loop: Detect → Explain → Alert → Prevent → Guide. This directly supports the hackathon theme of AI for cybersecurity awareness and the stated use case of identifying phishing messages and explaining security risks simply.

2. Problem Statement

Digital fraud frequently succeeds because victims do not recognize the warning signs quickly enough. Scam messages imitate banks, delivery companies, employers, government services and other trusted entities. They often create urgency, fear, curiosity or a reward incentive and push the victim toward a link, QR code, credential form or payment request.

The core problem is therefore not only threat detection. It is the gap between detection and human action: the user may know that something looks suspicious but still click because the message creates pressure.

CyberRaksha addresses three questions:

- Is this suspicious or potentially dangerous?
- Why is it dangerous, in language a beginner can understand?
- What should I do right now to avoid making the situation worse?

3. Objectives

- Detect common phishing/scam patterns using AI.
- Analyze text, screenshots, URLs and QR-code destinations through a single simple interface.
- Provide an explainable risk score and identify the main red flags.
- Generate simple, actionable advice rather than technical cybersecurity jargon.
- Trigger a high-priority emergency warning for high-confidence threats.
- Use a Safety Lock to prevent navigation to a suspicious destination within the controlled app flow.
- Support English, Hindi and Marathi explanations.
- Demonstrate responsible AI by showing confidence, evidence and safe fallback behavior.

4. Scope of the MVP

In Scope	Out of Scope
SMS/WhatsApp text pasted into the app	Full device-wide antivirus
Screenshot upload + OCR	Continuous background surveillance
URL analysis	Automatic deletion of user messages
QR decoding and destination analysis	Accessing private bank accounts
Scam category + risk score	Automatic financial transactions
Scam DNA / red-flag explanation	Guaranteed detection of every scam

Emergency alert	Forcing users to disconnect their entire phone from the internet
Safety Lock inside the app/browser flow	Uncontrolled OS-level network kill switch
English/Hindi/Marathi response	Large training/game platform

Important implementation clarification: “automatic internet connectivity off” should be implemented as a controlled Safety Lock rather than a device-wide Wi-Fi/mobile-data shutdown. A normal third-party mobile app cannot reliably or safely switch off another device’s network connectivity without privileged OS controls. For the hackathon prototype, the safer and technically demonstrable behavior is to block/stop the suspicious navigation, quarantine the URL inside the app, and show an emergency warning. If a managed-device or VPN-based extension is later used, network-level blocking can be added as a future version.

5. Core User Experience

Step	Behavior
1. User receives suspicious content	User opens CyberRaksha and pastes text, uploads a screenshot, enters a URL, or scans a QR code.
2. AI analyzes	The system extracts text/entities and evaluates phishing/scam indicators.
3. Risk decision	A confidence/risk score and category are generated.
4. Explain	The UI shows the top red flags and a simple explanation of the attack strategy.
5. Protect	If risk is high, Emergency Alert + Safety Lock are activated.
6. Guide	The user receives 2–4 immediate, safe actions in the selected language.

6. Key Features

6.1 AI Scam Analyzer

Analyzes suspicious content for common scam signals such as impersonation, urgency, threats, requests for credentials/OTP, unexpected payment requests, suspicious domains, fake rewards, fake jobs, fake KYC and delivery/courier scams.

6.2 Scam DNA — Signature Innovation

Scam DNA is an explainability layer that breaks a suspicious message into the tactics being used. Instead of only labeling a message as malicious, it identifies patterns such as authority impersonation, urgency, fear, financial pressure, suspicious URL, credential request and reward bait.

Signal	Example explanation
Authority impersonation	Pretends to be a bank, company, government office or known service.
Urgency	Creates a short deadline so the user acts before verifying.
Fear/threat	Claims the account will be blocked or a penalty will occur.
Credential request	Attempts to obtain passwords, OTPs, PINs or other sensitive data.
Suspicious destination	Uses an unfamiliar, misleading or mismatched URL/domain.
Payment pressure	Requests an unexpected payment or asks the user to transfer money.

6.3 Emergency Cyber Alert

When the risk is above a configured high-risk threshold, the interface switches to a prominent warning state. The prototype can use a red warning screen, vibration where supported, and an optional alert sound. The warning must clearly say what the user should not do and what safe action to take.

6.4 Safety Lock

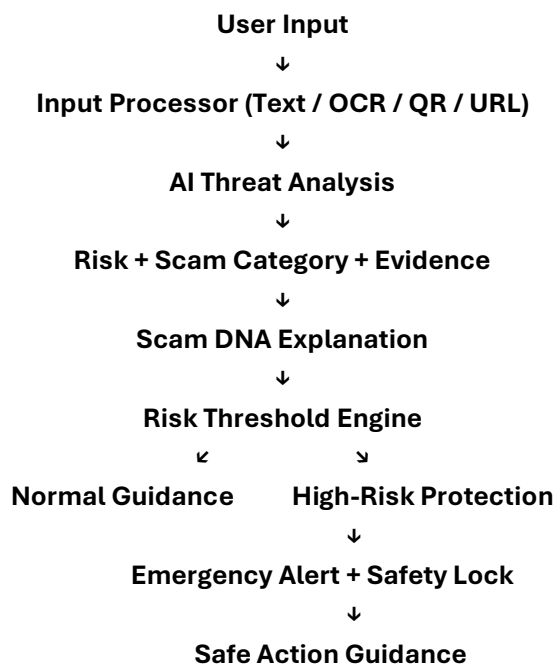
Safety Lock is the preventive innovation. If a suspicious URL is opened through CyberRaksha, the app does not continue to the destination when the threat is high. It stops the navigation, displays the threat evidence, and requires an explicit safe-action decision. This reduces the chance of an impulsive click.

6.5 Regional-Language Awareness

The same evidence-based explanation is generated in English, Hindi or Marathi. The language layer changes the communication, not the underlying security decision.

7. System Architecture

Recommended architecture:



7.1 Components

Component	Responsibility
Frontend	Upload/paste/scan, results, language selector, warning and Safety Lock UI.
Input Processor	OCR for screenshots, QR decoding, URL extraction and text normalization.
AI Analysis Service	Classifies scam intent/category and produces evidence-backed explanations.
Risk Engine	Combines AI output and deterministic security rules into a final risk level.
Safety Controller	Stops suspicious navigation and keeps the user inside a warning screen.
Guidance Generator	Creates concise safe actions in the selected language.

Logging/Analytics	Stores only minimal non-sensitive demo metrics; avoid raw personal messages by default.
-------------------	---

8. IBM Technology Integration

The project should visibly use IBM technology rather than merely mentioning IBM in the presentation. The exact services available to the team should be confirmed against the hackathon environment.

IBM layer	Proposed role
watsonx.ai / supported IBM foundation model	Scam reasoning, classification, explanation and multilingual response.
IBM Granite model where available	Use as the core language model for the awareness assistant, subject to hackathon availability.
IBM Cloud	Host backend/API and deploy the prototype.
watsonx.governance, where available	Track responsible-AI evaluation such as quality, explainability and safety.
IBM AI/agent capabilities, if provided by the event	Orchestrate the compact analysis → explanation → protection workflow.

Do not claim that an IBM service performs a capability unless the implemented prototype actually uses that service. During the build, record screenshots, API calls and architecture evidence for the IBM components used.

9. AI + Rule-Based Decision Logic

Use a hybrid design. The AI is responsible for understanding language and context; deterministic rules provide a safety boundary for high-risk actions. This is preferable to allowing a language model alone to decide whether to block navigation.

Layer	Example
AI interpretation	“This message impersonates a bank and pressures the user to complete KYC.”
Signals	Urgency=high, impersonation=high, suspicious URL=high, credential request=medium.
Risk calculation	Weighted score from AI signals + deterministic URL/security checks.
Policy	High risk → alert + Safety Lock; medium → caution; low → informational response.
Explanation	Show the top 3–5 evidence points used for the decision.
Fallback	If confidence is low, do not claim “safe”; say “Unable to verify—treat cautiously.”

9.1 Suggested risk levels

Level	Example	System behavior
Low	No meaningful scam indicators	Normal result + basic safety advice.
Medium	Some suspicious signals	Caution warning + verify independently.
High	Multiple strong scam signals	Emergency alert + Safety Lock + immediate actions.

10. Recommended Technology Stack

Layer	Recommended choice
Frontend	React for web prototype OR Flutter/React Native for mobile-style UI.

Backend	Python FastAPI or Node.js/Express.
AI	IBM watsonx.ai / Granite model available in the hackathon.
OCR	IBM-supported OCR/service if available, otherwise a lightweight OCR library.
QR	Standard QR decoding library.
URL checks	Safe parsing + domain/URL reputation signals; do not rely on a single source.
Database	Lightweight PostgreSQL/SQLite for prototype metadata; avoid storing raw sensitive content.
Deployment	IBM Cloud or the environment specified by the hackathon.

11. Step-by-Step Development Plan

Phase	Deliverable
Phase 1 — UI	Build the home screen: Paste Message, Upload Screenshot, Scan QR, Check URL, language selector.
Phase 2 — Text MVP	Connect one IBM AI model endpoint. Return scam category, risk, evidence and explanation.
Phase 3 — Screenshot	Add OCR, then send extracted text to the same analysis pipeline.
Phase 4 — URL/QR	Decode QR → extract URL → analyze URL signals → send context to the AI.
Phase 5 — Scam DNA	Convert evidence into a compact red-flag visualization.
Phase 6 — Safety Lock	Create an in-app browser/preview flow. High-risk URLs never proceed to the destination.
Phase 7 — Alert	Implement high-risk warning screen, optional sound/vibration and clear next actions.
Phase 8 — Languages	Generate or map explanations into English/Hindi/Marathi; verify with manually tested examples.
Phase 9 — Testing	Run known scam and benign examples; tune thresholds and reduce false positives.
Phase 10 — Demo	Prepare one strong end-to-end story and capture IBM technology evidence.

12. Hackathon Demonstration Scenario

Use one realistic fake-KYC example because it demonstrates the entire product without requiring many screens.

Demo action	Expected CyberRaksha response
Paste fake bank KYC message	AI identifies likely phishing/fake-KYC pattern.
Show result	High-risk score + category + Scam DNA.
Open suspicious link through CyberRaksha	Safety Lock stops navigation.
Emergency state	Large warning + optional sound/vibration.
Tap “Why?”	Simple explanation of urgency, impersonation and suspicious destination.
Switch to Marathi/Hindi	Same explanation and actions in selected language.
Choose “I already clicked”	Show safe incident-response guidance; do not pretend to perform banking actions automatically.

13. Suggested UI Screens

- Home: “Is this safe?” with four input choices.
- Analyzing screen: short progress state; avoid fake technical animations.
- Result screen: Risk score, scam type, top red flags and Scam DNA.

- Emergency screen: STOP / DO NOT PROCEED, reason, Safety Lock status, safe actions.
- Language selector: English / हिंदी / मराठी.
- History (optional): only local demo history or anonymized results; do not store sensitive content unnecessarily.

14. Testing Dataset & Evaluation

Create a small labeled test set for the demo and evaluation. Include both scam and benign examples so that the team can measure false positives as well as detection.

Category	Examples to include
Fake KYC/bank	Account block, KYC expiry, card suspension messages.
UPI/payment	Payment request, fake refund, collect-request/social engineering.
Job scam	Fake recruitment, registration fee, work-from-home payment.
Courier scam	Customs fee, parcel seizure, delivery payment.
Investment scam	Guaranteed returns, urgent investment offers.
Benign	Realistic normal bank/service/job messages without malicious indicators.
Multilingual	English, Hindi and Marathi variants of selected cases.

Measure: detection precision, recall, false-positive rate, explanation usefulness, response time, and successful Safety Lock activation for high-risk cases. For a hackathon, a small transparent evaluation is better than claiming unverified real-world accuracy.

15. Security, Privacy & Responsible AI

- Do not request or store OTPs, passwords, UPI PINs, full card numbers or banking credentials.
- Do not ask users to upload sensitive financial documents for the MVP.
- Mask personal phone numbers, email addresses and transaction identifiers in logs.
- Use HTTPS and secure API-key storage.
- Never label an item “safe” solely because the AI is uncertain. Use “unable to verify” when evidence is insufficient.
- Show why a high-risk decision was made.
- Use a deterministic safety policy around navigation rather than allowing the language model to directly execute risky actions.
- Provide official reporting/contact guidance rather than claiming CyberRaksha can recover money or file a complaint automatically.

16. Innovation Statement

CyberRaksha's innovation is the combination of explainable AI awareness and immediate preventive intervention in one compact user flow. The AI does not stop at “scam detected.” It identifies the manipulation strategy, communicates the danger simply, raises an emergency alert for high-confidence threats, and activates a Safety Lock before the user reaches the suspicious destination. This creates a practical bridge between cybersecurity awareness and safe user behavior.

17. Why CyberRaksha Can Stand Out

Typical “Is this safe?” tool	CyberRaksha
Returns safe/unsafe	Returns risk + evidence + explanation.
Detection-focused	Detection + awareness + prevention.
Generic warning	Emergency warning for high-risk cases.
User may still click	Safety Lock interrupts suspicious navigation in the controlled flow.
Technical language	Beginner-friendly English/Hindi/Marathi.
AI-only decision	AI understanding + deterministic safety policy.

18. Final Hackathon Pitch

“CyberRaksha is an AI-powered cybersecurity awareness assistant built around one simple question: ‘Is this safe?’ It analyzes suspicious messages, screenshots, links and QR codes, explains exactly why something is risky, and—when a high-confidence threat is detected—raises an emergency alert and activates a Safety Lock to stop the user from proceeding to the suspicious destination. Instead of only detecting scams, CyberRaksha helps users understand the attack and prevent the mistake before it becomes a loss.”

19. Proposed Hackathon Evaluation Positioning

Criterion	Target strength	How to demonstrate
Innovation	Very strong	Scam DNA + Safety Lock + Emergency Alert.
Theme alignment	Very strong	Directly maps to phishing identification and simple security education.
Feasibility	Strong	Focused MVP with one analysis pipeline and controlled browser/URL flow.
Impact	Strong	Addresses real user behavior and prevents impulsive interaction.
AI usage	Strong	IBM AI performs contextual classification/explanation; rules enforce safety.
Usability	Strong	One question, simple score, red flags and immediate action.
Responsible AI	Strong	Evidence, uncertainty handling, privacy and human-controlled actions.

20. Suggested Team Work Split

Role	Responsibilities
AI/Backend	watsonx integration, prompt/model flow, risk output, API.
Frontend	UI, result cards, Scam DNA, warning screen, language selector.
Security/Integration	URL/QR logic, Safety Lock, deterministic rules, privacy.
Testing/Presentation	Dataset, evaluation, demo script, architecture and pitch.

21. Final Build Checklist

- IBM AI service/model is actually integrated and visible in the implementation.
- User can paste a scam message and receive a risk result.
- User can upload a screenshot and analyze extracted text.
- User can check a URL and/or QR destination.
- Result includes scam type, risk level and evidence.
- Scam DNA is visually understandable.
- High-risk result triggers Emergency Alert.
- Suspicious navigation is stopped by Safety Lock inside the prototype.
- English/Hindi/Marathi explanations work for the selected demo cases.
- No passwords, OTPs, UPI PINs or unnecessary sensitive data are collected.
- False-positive and benign-message tests are included.
- One polished end-to-end demo is ready.

22. Final Product Definition

CyberRaksha should be built as a focused AI safety assistant, not as a full antivirus, banking security system or cybercrime portal. The winning demonstration is the moment a suspicious message is analyzed, the AI explains the manipulation in plain language, the system raises an emergency warning, and Safety Lock prevents the user from continuing to the suspicious destination. Every additional feature should be rejected if it weakens this central story.